Hasan Mohammed
08FEB2026

Draw It or Lose It

CS 230 Project Software Design Template

Version 1.1

Hasan Mohammed
08FEB2026

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 5

Document Revision History

Version	Date	Author	Comments
1.0	01/25/26	Hasan-Mohammed	Initial design draft. Basic backend implementation.
1.1	02/08/26	Hasan-Mohammed	Evaluation of client and server side requirements and consideration for major operating platforms.
1.2	02/22/26	Hasan-Mohammed	Finalizing document with design recommendations.

Instructions
Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Hasan Mohammed
08FEB2026

Executive Summary

The Gaming Room is a client that currently has a game, “Draw It or Lose It”, as one of their products. It is currently exclusively available to android devices. The Client wishes to universalize access to their game. The solution is to create a web-based version of the game and deploy it on server platforms. The current stage of product design is for the high level entities in the game and the roster system for the games, teams and players.

Requirements

- Active games instances in memory are limited to 1.
- Teams and players are present in each game instance.
- Teams, players, and games are all distinguished by unique identifier values.

Design Constraints

- The backend code must be implemented in Java for creating a scalable and flexible product. This is so because Java is agnostic to operating systems, so long as the hosting OS can run a Java Virtual Machine. (JVM).
- The frontend should be made with industry standard tools and development languages such as JavaScript frameworks to be compatible with most devices. This allows for cross device and platform compatibility.
- A universal set of Identifier values must be checked along with names of players, games, and teams to ensure no duplicates are made. This prevents data clutter and potential logic errors in the design.
- Active game instances are limited to 1 in memory. This must be accomplished using the Singleton design pattern in the backend.
- Web infrastructure must be scalable to adapt to fluctuating demand for the product. This prevents overloading resources to failure.

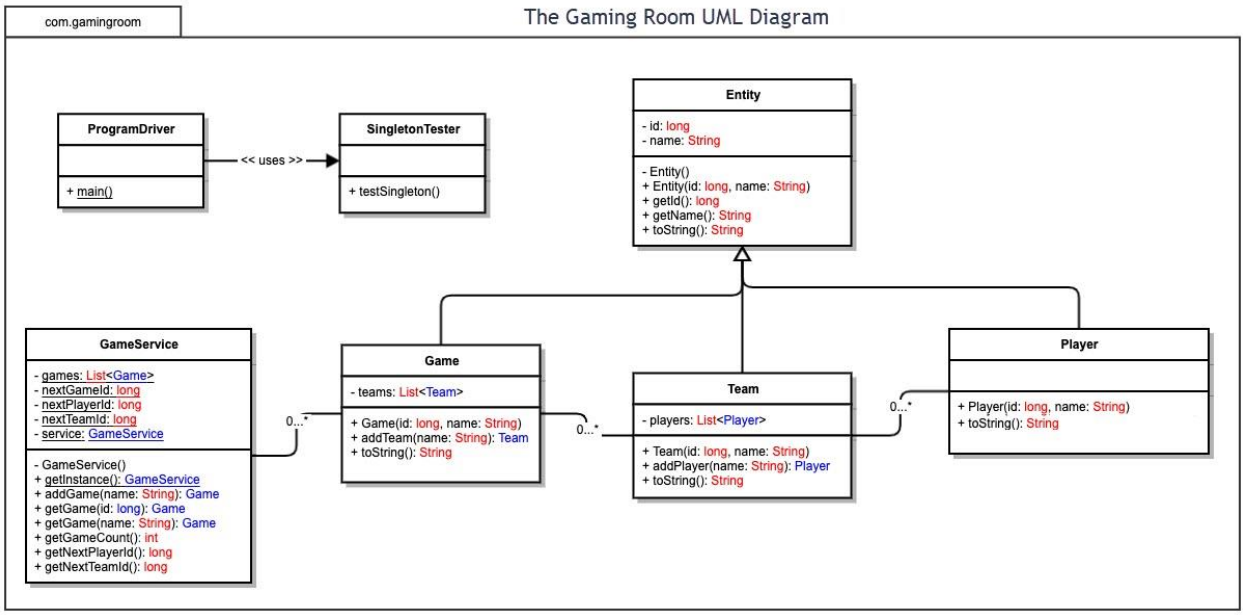
System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model
(NOTE: The Domain Model is for the basic backend implementation of the design constraints. Frontend and server deployment are handled separately.)

There is a parent Entity class that contains identifying information in the name and ID members. It has mutators, accessors and a toString override. Three classes, Team, Game and Player inherit from Entity. This allows for ID tracking of all objects for detecting duplicates and easy searching for our design constraint. Apart from constructors taking in their parent’s parameters, the Game class has an addTeam function to create a team if it doesn't exist, and return an existing one if the provided name argument matches with it. The exact same logic applies to the Team class with its addPlayer function, looking into

a list of players instead of teams. The Player class has no unique method or member variable apart from its overridden toString method. Team and Game also have their own toString overrides. Finally, the three child classes are employed by a GameService class that adopts the Singleton pattern. This is so with its private constructor and a singular pre-instantiated instance member called service. It contains a list of games (called games) and the nextGameId, nextPlayerid, nextTeamId for assignment to newly instantiated objects. An important getInstance method allows for a public handle on the GameService class. The rest are for adding and retrieving game objects, along with getting the next Id for players, games, and Teams for their assignment.



Evaluation

Evaluation will be done according to the available operating platforms suitable for hosting "Draw It or Lose It". Namely, they are Windows, Mac OS X, and Linux. The server version of these operating systems will be discussed for evaluation. Advantages and drawbacks will be grouped off the server and client implementations.

Server

Windows: Comes with built-in, proprietary software by Microsoft for quality-of-life improvements. The latest version of Windows Server 2025 comes installed with a modern terminal, a system performance diagnostic tool, Dtrace, and the WinGet package manager. Additionaly, there is a native integration with other Microsoft services like Azure that allow for flexibility of use. The drawbacks are mainly financial. Just one license for a Windows Server 2025 edition will cost around $1100 USD. There is a separate cost for each individual client using it that goes for $50 USD per unit. This is a steep price to pay compared to other available platforms.

Linux: Linux is a UNIX-based operating system that is in the same family as Max OS X. It offers a completely free licensing plan. There is no centralized vendor for servers since the entire operating system is open source and free to use. Added to the lack of financial cost, Linux is not a monolithic

operating system. There are numerous different versions of the OS, called distros, that are modular in nature. This allows for extreme versatility. The ability to suit the OS to software requirements allows for minimal overhead to complicate things. Additionally, Linux is adopted by the industry for hosting solutions, meaning there will be a plethora of compatible tooling to pick from. Its main downfall is from its flexibility. Since there are many distros to choose from, expertise is required to effectively leverage the OS. Its lack of uniformity like Windows will complicate matters when more employees are involved in the hosting process.

Mac OS X: The main difference between this UNIX-based operating platform and Linux is of its restricted, proprietary nature. This allows its developers/maintainers to have a competitive level of security by constantly patching any vulnerabilities a cyber attacker might exploit. However, this feature is shared with Windows. The difference, again, lies with the fact that it is a UNIX-based OS. This allows existing tech industry workers to switch from Linux with relative ease. The drawbacks start with the OS being completely restricted to its own brand of hardware. Hosting a server on a Mac will have heavy vendor costs. Especially so, if the hardware was bought for self-hosting. Finally, as far as the Internet has listed, there is no modern Mac OS X server version that is being actively maintained today. This is a massive drawback as a choice of hosting platform.

Mobile: There are no significant benefits of mobile hosting over the other operating platforms due to the intent and design of mobile devices to be functioning as clients, not servers. They are severely limited by hardware and operating system software. The two major OS choices, Apple's IOS and Google's Android, offer little compared to the other platforms. IOS is completely proprietary and restricts any modification of the OS. It is meant to be used as an Apple client for consumers. Android is more flexible since it is based off a Linux platform. However, it is still proprietary by nature. There is some flexibility in the ability to install custom software. However, this is as far as the benefits go for mobile devices.

Client

Client side development must use RESTful communication with the server backend using HTTP. This will allow the communication and display of information to be platform-agnostic. Expertise must be focused on web development to suit that requirement. The choice of HTML presentation should be compatible with the platform specific browsers such as safari or microsoft edge.

Development tools

Tools needed for development should be open source, free to use, and widely adopted. IntelliJ Idea and Eclipse are consistent choices for Java development for the backend. Visual Studio Code could also be used as a substitute since it can be configured to work with any language. Licensing costs should be kept to zero if these are used. The DropWizard framework could be used for RESTful communication with the backend. Necessary costs and tools could most likely be used for cloud hosting services for a Linux server to run the java backend. Multiple teams are not likely needed since the entire project can be centered around HTML/REST java. The same development team can have their priorities moved towards maintenance and deployment after the game is finished.

Recommendations:

As mentioned previously in the server-side evaluation, a Linux server would be the best choice for deployment and/or development. Considering that cost is tied directly to the digital resources in demand, such as RAM or processor speed, this operating platform is modular and can prune unnecessary features from itself for maximum efficiency. This allows for getting more digital resources out of the investment given. Adding the fact that a good amount of Linux server platforms demands less overhead than windows or Mac environments, the contrast is sharp. Some relevant details to include are that the OS should be 64-bit and should provide enough modularity. This is most effective with Debian-based distros such as Debian itself or Ubuntu Server. The choice is mainly due to ease of use and out-of-box performance that beats windows in raw network packet throughput. Finally, Linux is completely free to use and doesn't require licensing costs like Windows. As for storage, the industry standard for a library of media is Amazon S3. This is simply an extremely scalable cloud storage service that is universally available. That can be used to store the collection of images to load on the clients. To improve performance, a CDN (Content Delivery Network) can be used to store local copies of the library of images in nearby servers for loading. However, this would incur additional cost. If local storage is viable, an ext4 type of file system is appropriate since its much faster with read/write operations for storing game session state for backups. NTFS is also available but is much slower at the optional advantage of being universally interfaced. This is only ideal if the storage is meant to be switched between different operating platforms. Memory management will be simply managed by the Singleton pattern of application design. Which means that instances of each game app in memory will be limited to one. This simplified the process of game sessions by ensuring that there was a central store for data to be modified in memory. For the design of distributed systems and networks, a simple server-client model will suffice with small caveats. One is that there should be an intermediate proxy between the client and server that maintains a cache containing session information. This design accommodates for network outages. Adding the concept of CDN is a potentially additional responsibility for that proxy. Finally, most of the security should be centered around the server. This is so because the app will be developed in Java, and therefore, lacks memory vulnerabilities via the garbage collector feature. Data traversing the web will be fully encrypted via TLS (Transport Layer Security) and will use RESTful communication. Each client will be required to authenticate with the OAuth API built into the app. Once authenticated, the server will then respond to the client with their appropriate information and game state.